

SETUP AND ACCESS

Authentication

Log in to Claude Code and configure authentication for individuals, teams, and organizations.

Claude Code supports multiple authentication methods depending on your setup. Individual users can log in with a Claude.ai account, while teams can use Claude for Teams or Enterprise, the Claude Console, or a cloud provider like Amazon Bedrock, Google Vertex AI, or Microsoft Foundry.

Log in to Claude Code

After installing Claude Code, run `cclaude` in your terminal. On first launch, Claude Code opens a browser window for you to log in.

If the browser doesn't open automatically, press `c` to copy the login URL to your clipboard, then paste it into your browser.

If your browser shows a login code instead of redirecting back after you sign in, paste it into the terminal at the `Paste code here if prompted` prompt. This happens when the browser can't reach Claude Code's local callback server, which is common in WSL2, SSH sessions, and containers.

You can authenticate with any of these account types:

- **Claude Pro or Max subscription:** log in with your Claude.ai account. Subscribe at claude.com/pricing.
- **Claude for Teams or Enterprise:** log in with the Claude.ai account your team admin invited you to.
- **Claude Console:** log in with your Console credentials. Your admin must have invited you first.
- **Cloud providers:** if your organization uses [Amazon Bedrock](#), [Google Vertex AI](#), or [Microsoft Foundry](#), set the required environment variables before running `cclaude`. No browser login is needed.

To log out and re-authenticate, type `/logout` at the Claude Code prompt.

If you're having trouble logging in, see [authentication troubleshooting](#).

Set up team authentication

For teams and organizations, you can configure Claude Code access in one of these ways:

- **Claude for Teams or Enterprise**, recommended for most teams
- **Claude Console**
- **Amazon Bedrock**
- **Google Vertex AI**
- **Microsoft Foundry**

Claude for Teams or Enterprise

Claude for Teams and **Claude for Enterprise** provide the best experience for organizations using Claude Code. Team members get access to both Claude Code and Claude on the web with centralized billing and team management.

- **Claude for Teams:** self-service plan with collaboration features, admin tools, and billing management. Best for smaller teams.
- **Claude for Enterprise:** adds SSO, domain capture, role-based permissions, compliance API, and managed policy settings for organization-wide Claude Code configurations. Best for larger organizations with security and compliance requirements.

1 Subscribe

Subscribe to **Claude for Teams** or contact sales for **Claude for Enterprise**.

2 Invite team members

Invite team members from the admin dashboard.

3 Install and log in

Team members install Claude Code and log in with their Claude.ai accounts.

Claude Console authentication

For organizations that prefer API-based billing, you can set up access through the Claude Console.

1 Create or use a Console account

Use your existing Claude Console account or create a new one.

2 Add users

You can add users through either method:

- Bulk invite users from within the Console: Settings -> Members -> Invite
- Set up SSO

3 Assign roles

When inviting users, assign one of:

- **Claude Code** role: users can only create Claude Code API keys
- **Developer** role: users can create any kind of API key

4 Users complete setup

Each invited user needs to:

- Accept the Console invite
- Check system requirements
- Install Claude Code
- Log in with Console account credentials

Cloud provider authentication

For teams using Amazon Bedrock, Google Vertex AI, or Microsoft Foundry:

1 Follow provider setup

Follow the Bedrock docs, Vertex docs, or Microsoft Foundry docs.

2 Distribute configuration

Distribute the environment variables and instructions for generating cloud credentials to your users. Read more about how to [manage configuration here](#).

3 Install Claude Code

Users can [install Claude Code](#).

Credential management

Claude Code securely manages your authentication credentials:

- **Storage location:**
 - On macOS, credentials are stored in the encrypted macOS Keychain.
 - On Linux, credentials are stored in `~/.claude/.credentials.json` with file mode `0600`.
 - On Windows, credentials are stored in `%USERPROFILE%\.claude\.credentials.json` and inherit the access controls of your user profile directory, which restricts the file to your user account by default.
 - If you've set the `CLAUDE_CONFIG_DIR` environment variable on Linux or Windows, the `.credentials.json` file lives under that directory instead.
 - Claude Code manages `.credentials.json` through `/login` and `/logout`. To route requests through a custom API endpoint, set the [ANTHROPIC_BASE_URL](#) environment variable instead.
- **Supported authentication types:** Claude.ai credentials, Claude API credentials, Azure Auth, Bedrock Auth, and Vertex Auth.
- **Custom credential scripts:** the [apiKeyHelper](#) setting can be configured to run a shell script that returns an API key.
- **Refresh intervals:** by default, `apiKeyHelper` is called after 5 minutes or on HTTP 401 response. Set `CLAUDE_CODE_API_KEY_HELPER_TTL_MS` environment variable for custom refresh intervals.
- **Slow helper notice:** if `apiKeyHelper` takes longer than 10 seconds to return a key, Claude Code displays a warning notice in the prompt bar showing the elapsed time. If you see this notice regularly, check whether your credential script can be optimized.

`apiKeyHelper`, `ANTHROPIC_API_KEY`, and `ANTHROPIC_AUTH_TOKEN` apply to the CLI and the surfaces that wrap it, including the VS Code extension, the Agent SDK, and GitHub Actions. Claude Desktop and cloud sessions do not call `apiKeyHelper` or read these environment variables: they

use OAuth, except desktop sessions running an organization-distributed third-party inference configuration, which authenticate with that configuration's credential.

Authentication precedence

When multiple credentials are present, Claude Code chooses one in this order:

1. Cloud provider credentials, when `CLAUDE_CODE_USE_BEDROCK` , `CLAUDE_CODE_USE_VERTEX` , or `CLAUDE_CODE_USE_FOUNDRY` is set. See third-party integrations for setup.
2. `ANTHROPIC_AUTH_TOKEN` environment variable. Sent as the `Authorization: Bearer` header. Use this when routing through an LLM gateway or proxy that authenticates with bearer tokens rather than Anthropic API keys.
3. `ANTHROPIC_API_KEY` environment variable. Sent as the `X-API-Key` header. Use this for direct Anthropic API access with a key from the Claude Console. In interactive mode, you are prompted once to approve or decline the key, and your choice is remembered. To change it later, use the “Use custom API key” toggle in `/config` . In non-interactive mode (`-p`), the key is always used when present.
4. `apiKeyHelper` script output. Use this for dynamic or rotating credentials, such as short-lived tokens fetched from a vault.
5. `CLAUDE_CODE_OAUTH_TOKEN` environment variable. A long-lived OAuth token generated by `claude setup-token` . Use this for CI pipelines and scripts where browser login isn't available.
6. Subscription OAuth credentials from `/login` . This is the default for Claude Pro, Max, Team, and Enterprise users.

If you have an active Claude subscription but also have `ANTHROPIC_API_KEY` set in your environment, the API key takes precedence once approved. This can cause authentication failures if the key belongs to a disabled or expired organization. Run `unset ANTHROPIC_API_KEY` to fall back to your subscription, and check `/status` to confirm which method is active.

Claude Code on the Web always uses your subscription credentials. `ANTHROPIC_API_KEY` and `ANTHROPIC_AUTH_TOKEN` in the sandbox environment do not override them.

Generate a long-lived token

For CI pipelines, scripts, or other environments where interactive browser login isn't available, generate a one-year OAuth token with `claude setup-token` :

```
claude setup-token
```

The command walks you through OAuth authorization and prints a token to the terminal. It does not save the token anywhere; copy it and set it as the `CLAUDE_CODE_OAUTH_TOKEN` environment variable wherever you want to authenticate:

```
export CLAUDE_CODE_OAUTH_TOKEN=your-token
```

This token authenticates with your Claude subscription and requires a Pro, Max, Team, or Enterprise plan. It is scoped to inference only and cannot establish Remote Control sessions.

Bare mode does not read `CLAUDE_CODE_OAUTH_TOKEN`. If your script passes `--bare`, authenticate with `ANTHROPIC_API_KEY` or an `apiKeyHelper` instead.